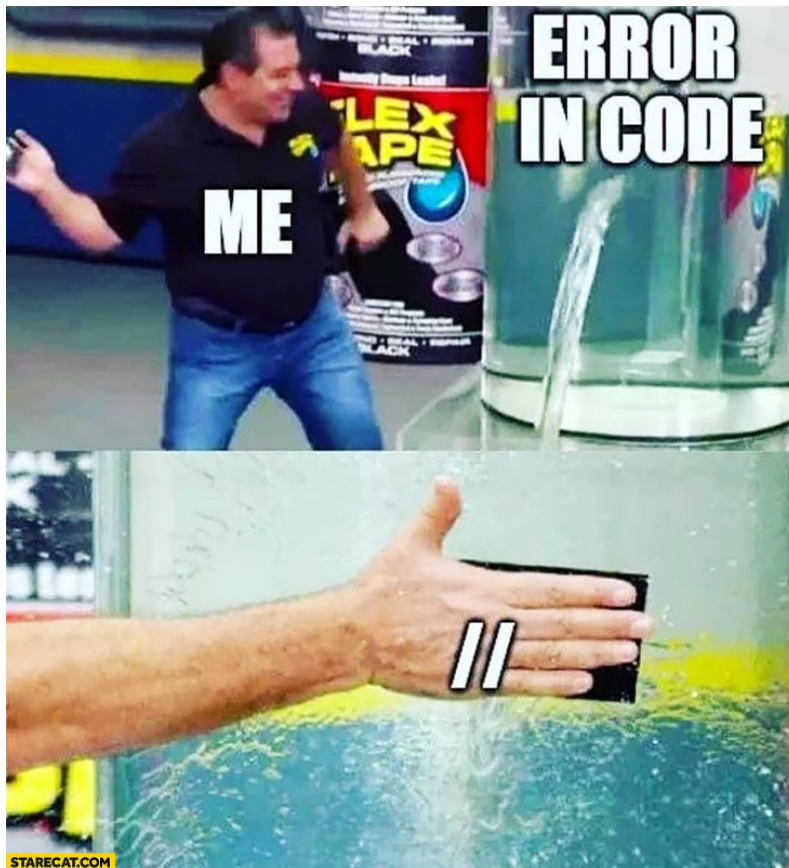


Lab 7: Virtual File System

Contents

- Introduction
- Goals of this lab
- Background
- Basic Exercises
- Advanced Exercises



Introduction

A file system manages data in storage mediums. Each file system has a specific way to store and retrieve the data. Hence, a virtual file system (VFS) is common in general-purpose OS, providing a unified interface for all file systems.

[Skip to main content](#)

In this lab, you'll implement a VFS interface for your kernel, and a memory-based file system (tmpfs) that mounts as the root file system, you'll also implement special file for uart and framebuffer.

Goals of this lab

- Understand how VFS interface works.
- Understand how to set up a root file system.
- Understand how to operate on files.
- Understand how to mount a file system and look up a file across file systems.
- Understand how special file works.

Background

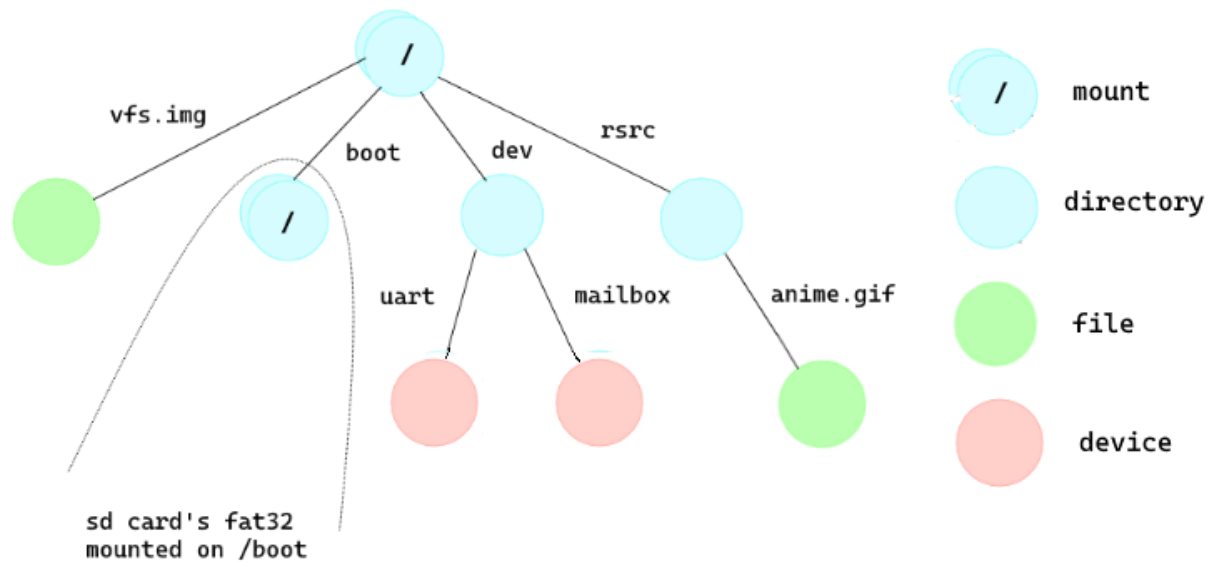
Tree Structure

A file system is usually hierarchical, and a tree is a suitable data structure to represent it.

Each node represents an entity such as a file or directory in the file system, and each edge has its name, which is stored in the directory.

Concatenating all the edges' name on the path generates a pathname, which VFS can parse and traverse from one node to another.

Example graph:



Terminology

File System

In this documentation, a file system refers to a concrete file system type such as tmpfs, FAT32, etc. And the virtual file system will be shortened as VFS.

Vnode

We call a node in a VFS tree a vnode, which is an abstract class that provides an unified interface, the underlying file systems should implement the methods and create the instance.

Component Name

A pathname delimits each component name by '/'.

File Handle

Files can be opened simultaneously. You should maintain a data structure that keeps information such as position of next R/W operation for each opened file. We call this data structure file handle.

Basic Exercises

For basic exercises, you'll need to create an VFS interface first.

We've provide an example code for you, and it's up to you to re-design or modify the code.

Note

1. There's no constraint on the design of VFS api, as long as there are no hard coded method for any file system (VFS is an generalize interface).
2. The following steps are just recommendations, you don't have to follow them.
3. For tmpfs, you can assume that component name won't excced 15 characters, and at most 16 entries for a directory. and at most 4096 bytes for a file.
4. For VFS the max pathanme length is 255.
5. No need to support remove, rmdir, unlink ...
6. For file descriptor, max open fd is 16, so $fd < 16$ if your fd number is reusable.

```
struct vnode {
    struct mount* mount;
    struct vnode_operations* v_ops;
    struct file_operations* f_ops;
    void* internal;
};

// file handle
struct file {
    struct vnode* vnode;
    size_t f_pos; // RW position of this file handle
    struct file_operations* f_ops;
    int flags;
};

struct mount {
    struct vnode* root;
    struct filesystem* fs;
};

struct filesystem {
    const char* name;
    int (*setup_mount)(struct filesystem* fs, struct mount* mount);
};

struct file_operations {
    int (*write)(struct file* file, const void* buf, size_t len);
    int (*read)(struct file* file, void* buf, size_t len);
    int (*open)(struct vnode* file_node, struct file** target);
};
```

[Skip to main content](#)

```

};

struct vnode_operations {
    int (*lookup)(struct vnode* dir_node, struct vnode** target,
                  const char* component_name);
    int (*create)(struct vnode* dir_node, struct vnode** target,
                  const char* component_name);
    int (*mkdir)(struct vnode* dir_node, struct vnode** target,
                  const char* component_name);
};

struct mount* rootfs;

int register_filesystem(struct filesystem* fs) {
    // register the file system to the kernel.
    // you can also initialize memory pool of the file system here.
}

int vfs_open(const char* pathname, int flags, struct file** target) {
    // 1. Lookup pathname
    // 2. Create a new file handle for this vnode if found.
    // 3. Create a new file if O_CREAT is specified in flags and vnode not found
    // lookup error code shows if file exist or not or other error occurs
    // 4. Return error code if fails
}

int vfs_close(struct file* file) {
    // 1. release the file handle
    // 2. Return error code if fails
}

int vfs_write(struct file* file, const void* buf, size_t len) {
    // 1. write len byte from buf to the opened file.
    // 2. return written size or error code if an error occurs.
}

int vfs_read(struct file* file, void* buf, size_t len) {
    // 1. read min(len, readable size) byte to buf from the opened file.
    // 2. block if nothing to read for FIFO type
    // 2. return read size or error code if an error occurs.
}

int vfs_mkdir(const char* pathname);
int vfs_mount(const char* target, const char* filesystem);
int vfs_lookup(const char* pathname, struct vnode** target);
...

```

Basic Exercise 1 - Root File System - 35%

In this part, you'll need to implement tmpfs which follows the VFS interface, setup tmpfs as the root file system.

[Skip to main content](#)

File System Registration

Since each file system has its own initialization method, the VFS should provide interface for each file system to register. Then, users can mount the file system by specifying the name.

Mounting File System

The provided code uses **struct mount** to represent a mounted file system, VFS should provide an api for mounting a file system to a mount point.

The root file system is at the top of the VFS tree, you should mount tmpfs at rootfs, at this point, lookup might not be available, you can call `setup_mount` directly to mount it.

Root directory's vnode

Each mounted file system has its own root vnode. You should create the root vnode during the mount setup.

The internal representation of each filesystem's vnode may differ, you can use `vnode.internal` to point to it.

Open Method

It opens the vnode regardless of the underlying file system and file type, and creates a file handle for the file.

Close Method

close and release the file handle.

Read Method

Given the file handle, VFS calls the corresponding read method to read the file starting from `f_pos`, then updates `f_pos` after read. (or not if it's a special file)

Note that `f_pos` should not exceed the file size. Once a file read reaches the end of

[Skip to main content](#)

Write Method

Given the file handle, VFS calls the corresponding write method to write the file starting from `f_pos`, then updates `f_pos` and size after write. (or not if it's a special file) Returns size written or error code on error.

Lookup Method

File system iterates through directory entries and compares the component name to find the target file. Then, it passes the file's vnode to the VFS if it finds the file.

Create Method

create an regular file on underlying file system, should fail if file exist. Then passes the file's vnode back to VFS.

Basic Exercise 2 - Multi-level VFS - 15%

In this part, your VFS should be able to

- create subdirectories
- mount file systems on directories
- look up an pathname

Mkdir Method

create a directory on underlying file system, same as creating a regular file.

Mounting Another File System

same as mounting the root file system, except, you should be able to mount on any vnode.

Pathname Lookup

VFS api takes pathname as argument (absolute path `"/"` for now). you need to lookup the

[Skip to main content](#)

Also, the lookup should be able to cross mounting point. For mounted vnode, VFS should go to the mounted file system's root vnode.

pseudo code, this code doesn't show crossing of mount point or relative pathname.

```
int vfs_lookup(const char* pathname, struct vnode** target) {
    auto vnode_itr = rootfs->root;
    for (component_name : pathname) {
        auto next_vnode;
        auto ret = vnode_itr->v_ops->lookup(vnode_itr, next_vnode, component_name);
        if (ret != 0) {
            return ret;
        }
        vnode_itr = next_vnode;
    }
    *target = vnode_itr;
    return 0;
}
```

Basic Exercise 3 - Multitask VFS - 15%

In this part, you need to implement

- current working directory
- file descriptor table
- system calls for VFS

Current Working Directory

Each task may have different current working directory and root directory, you need to keep that information in your task struct, and VFS should traverse vnode base on those information.

You'll need to support relative path lookup, `"`, `"."`, `".."`, and if the root vnode is mounted on a vnode, VFS should go to the mounted vnode, other wise, `".."` behaves like

`"."`

You also need to support change directory, to modify current working directory.

File Descriptor Table

Each process should have a file descriptor table to bookkeep the opened files. When the user opens a file, the kernel creates a file handle in the table and returns the index (file descriptor) to the user. After that, the user can pass the file descriptor to the kernel to get the file handle. Then, the kernel calls the corresponding VFS API using the file handle and return the result to the user.

System Calls for VFS

You'll need to provide the following system calls

```
#define O_CREAT 00000100
// syscall number : 11
int open(const char *pathname, int flags);

// syscall number : 12
int close(int fd);

// syscall number : 13
// remember to return read size or error code
long write(int fd, const void *buf, unsigned long count);

// syscall number : 14
// remember to return read size or error code
long read(int fd, void *buf, unsigned long count);

// syscall number : 15
// you can ignore mode, since there is no access control
int mkdir(const char *pathname, unsigned mode);

// syscall number : 16
// you can ignore arguments other than target (where to mount) and filesystem
int mount(const char *src, const char *target, const char *filesystem, unsigned flags);

// syscall number : 17
int chdir(const char *path);
```

Basic Exercise 4 - /initramfs - 15%

In this part, you need to make initramfs as an read only file system which follows the VFS interface, and mount on `"/initramfs"`.

You might need to create supporting data structure for initramfs at registration (or mount), then create directory `"/initramfs"` on root file system, then mount on it.

All method like write, create, mkdir, should fail on this file system.

Advanced Exercises

Special File

A file in VFS can also represent a device. In order to create a special file, the normal way is to use mknod, which creates a special file with specified device driver, and file_operation points to driver's method.

Device File Registration

A device can register itself to the VFS in its setup. The VFS assigns the device a unique device id. Then the device can be recognized by the VFS.

Mknod

A user can use the device id to create a device file in a file system.

After the device file created, the VFS uses the device id to find the device driver. Next, the driver initializes the file with its method. Then, the user can read/write the file to access the device.

1. mkdir `"/dev"`
2. mknod `"/dev/uart"`
3. mknod `"/dev/framebuffer"`

Alternative

There's also other way to hack this (which is weird, but it works). As long as it's not hard coded on VFS, and it works fine with VFS api.

- Creat devfs, which have hard coded device file, and mount on `"/dev"`
- 1. mkdir `"/dev"`

[Skip to main content](#)

2. mount "devfs" on "/dev"
- Creat uartfs and mount on "/dev/uart"
 1. mkdir "/dev"
 2. mkdir "/dev/uart"
 3. mount "uartfs" on "/dev/uart"
 4. mkdir "/dev/framebuffer"
 5. mount "framebufferfs" on "/dev/framebuffer"

Advanced Exercises 1 - /dev/uart - 15%

You need to create a device file at "/dev/uart" for your UART device as the console. R/W to this file is same as uartread / uartwrite.

You also need to open this file as stdin (fd 0), stdout (fd 1), and stderr (fd 2), for user process

```
// there should be output on terminal
write(1, "hello world\n", 12);
```

Advanced Exercises 2 - /dev/framebuffer - 15%

In previous lab, we use mbox_call to establish framebuffer, and mapped VC memory for user in order to use framebuffer. Which means a system call and memory map is needed for every device.

In this part, we convert it to a write only special device at "/dev/framebuffer"

You need to initialize the framebuffer using mailbox at mknod (or mount or ioctl, we don't care), and when written, write directly to framebuffer (lfb + f_pos).

Also you need to support lseek64 system call (to write framebuffer again without reopen) and ioctl system call (to query framebuffer info), which are also VFS api using underlying method.

```
// syscall number : 18
// you only need to implement seek set
# define SEEK_SET 0
long lseek64(int fd, long offset, int whence);

// syscall number : 19
int ioctl(int fd, unsigned long request, ...);
```

[Skip to main content](#)

```
// if it works with default value, you can ignore this syscall
ioctl(fb, 0, &fb_info)
// remember to translate userspace address to kernel space
```

The following code will be in test program

```
int fb = open("/dev/framebuffer", O_WRONLY);

struct framebuffer_info {
    unsigned int width;
    unsigned int height;
    unsigned int pitch;
    unsigned int isrgb;
};

framebuffer_info fb_info { 1024, 768, 4096, 1};

ioctl(fb, 0, &fb_info);

while(true) {
    ...
    lseek64(fb, offset, SEEK_SET)
    ...
    write(fb, color, 4);
    ...
}
```

The following code is for mailbox initialize used in previous lab.

```
#define MBOX_REQUEST 0
#define MBOX_CH_PROP 8
#define MBOX_TAG_LAST 0

unsigned int __attribute__((aligned(16))) mbox[36];
unsigned int width, height, pitch, isrgb; /* dimensions and channel order */
unsigned char *lfb; /* raw frame buffer address */

mbox[0] = 35 * 4;
mbox[1] = MBOX_REQUEST;

mbox[2] = 0x48003; // set phy wh
mbox[3] = 8;
mbox[4] = 8;
mbox[5] = 1024; // FrameBufferInfo.width
mbox[6] = 768; // FrameBufferInfo.height

mbox[7] = 0x48004; // set virt wh
mbox[8] = 8;
mbox[9] = 8;
mbox[10] = 1024; // FrameBufferInfo.virtual_width
mbox[11] = 768; // FrameBufferInfo.virtual_height

mbox[12] = 0x48009; // set virt offset
```

[Skip to main content](#)

```

mbox[15] = 0; // FrameBufferInfo.x_offset
mbox[16] = 0; // FrameBufferInfo.y_offset

mbox[17] = 0x48005; // set depth
mbox[18] = 4;
mbox[19] = 4;
mbox[20] = 32; // FrameBufferInfo.depth

mbox[21] = 0x48006; // set pixel order
mbox[22] = 4;
mbox[23] = 4;
mbox[24] = 1; // RGB, not BGR preferably

mbox[25] = 0x40001; // get framebuffer, gets alignment on request
mbox[26] = 8;
mbox[27] = 8;
mbox[28] = 4096; // FrameBufferInfo.pointer
mbox[29] = 0; // FrameBufferInfo.size

mbox[30] = 0x40008; // get pitch
mbox[31] = 4;
mbox[32] = 4;
mbox[33] = 0; // FrameBufferInfo.pitch

mbox[34] = MBOX_TAG_LAST;

// this might not return exactly what we asked for, could be
// the closest supported resolution instead
if (mbox_call(MBOX_CH_PROP, mbox) && mbox[20] == 32 && mbox[28] != 0) {
    mbox[28] &= 0x3FFFFFFF; // convert GPU address to ARM address
    width = mbox[5]; // get actual physical width
    height = mbox[6]; // get actual physical height
    pitch = mbox[33]; // get number of bytes per line
    isrgb = mbox[24]; // get the actual channel order
    lfb = (void *)((unsigned long)mbox[28]);
} else {
    puts("Unable to set screen resolution to 1024x768x32\n");
}

```

Test

put this  [user program](#) in initramfs.cpio

```

// note that your exec should be using VFS api
exec("/initramfs/vfs1.img")

```

fork will validate framebuffer, and **vfs** will validate the rest

< [Previous](#)
[Lab 6: Virtual Memory](#)

[Lab 8: Non-volatile Storage](#) > [Next](#)